

Node.js Backend Patterns for Full-Stack JS Engineers

This covers Node.js-specific patterns, differences from browser JS, and server-side concerns.

1) JS vs Node.js: Key Differences

1.1 Execution Environment

Browser:

- Single-threaded event loop.
- DOM and Web APIs.
- Same-origin policy (CORS).
- localStorage, sessionStorage.

Node.js:

- Single-threaded event loop (main).
- Worker Threads for CPU tasks.
- libuv thread pool for I/O.
- File system, Network, Process APIs.
- No DOM.

1.2 Module System

Browser (ES6 modules):

```
import fs from "fs"; // ❌ Error: no fs in browser
import React from "react";
export default App;
```

Node.js (CommonJS or ES6):

```
const fs = require("fs");
const path = require("path");

module.exports = myFunction;

// or ES6
import fs from "fs";
export default myFunction;
```

1.3 Global Objects

Browser:

```
window, document, localStorage, navigator
```

Node.js:

```
global, process, __filename, __dirname, Buffer, setImmediate, clearImmediate
```

2) Process and Threads in Node.js

2.1 Process Object

```
console.log(process.pid); // process ID
console.log(process.cwd()); // current working directory
console.log(process.env.NODE_ENV); // environment variables

process.on("uncaughtException", (err) => {
  console.error("Uncaught Exception:", err);
  process.exit(1);
});

process.on("unhandledRejection", (reason, promise) => {
  console.error("Unhandled Rejection at:", promise, "reason:", reason);
});
```

2.2 Worker Threads for CPU Tasks

Problem:

```
// Blocks event loop for 5 seconds
function fibonacci(n) {
  if (n <= 1) return n;
  return fibonacci(n - 1) + fibonacci(n - 2);
}

app.get("/fib/:n", (req, res) => {
  const result = fibonacci(50); // blocks all requests
  res.json({ result });
});
```

Solution with Worker Threads:

```
const { Worker } = require("worker_threads");
const path = require("path");

app.get("/fib/:n", (req, res) => {
  const worker = new Worker(path.join(__dirname, "fib-worker.js"));

  worker.on("message", (result) => {
    res.json({ result });
  });

  worker.postMessage(parseInt(req.params.n));
});

// fib-worker.js
const { parentPort } = require("worker_threads");
```

```
parentPort.on("message", (n) => {
  const result = fibonacci(n);
  parentPort.postMessage(result);
});
```

Interview expectation:

- know when to use worker threads for CPU-intensive tasks.

2.3 Child Process Spawning

```
const { spawn, exec } = require("child_process");

// spawn: large data streams
const ls = spawn("ls", ["-la"]);
ls.stdout.on("data", (data) => {
  console.log(`stdout: ${data}`);
});

// exec: run shell commands
exec("ls -la", (error, stdout, stderr) => {
  if (error) console.error(error);
  console.log(stdout);
});
```

3) Streams and Large File Handling

3.1 Readable Stream

```
const fs = require("fs");

const stream = fs.createReadStream("large-file.txt", {
  encoding: "utf8",
  highWaterMark: 16 * 1024 // 16KB chunks
});

stream.on("data", (chunk) => {
  console.log("received chunk of size:", chunk.length);
});

stream.on("end", () => {
  console.log("stream ended");
});

stream.on("error", (err) => {
  console.error(err);
});
```

3.2 Pipe: Connect Streams

```
// Copy file efficiently
fs.createReadStream("source.txt")
  .pipe(fs.createWriteStream("dest.txt"));

// Transform and save
fs.createReadStream("data.json")
  .pipe(transform())
  .pipe(fs.createWriteStream("output.json"));
```

3.3 Transform Stream

```
const { Transform } = require("stream");
const csv = require("csv-parser");

fs.createReadStream("data.csv")
  .pipe(csv())
  .pipe(new Transform({
    objectMode: true,
    transform(record, encoding, callback) {
      record.processed = true;
      callback(null, JSON.stringify(record));
    }
  })))
  .pipe(fs.createWriteStream("output.json"));
```

Interview point:

- streams prevent loading entire files in memory.

4) Event-Driven Architecture

4.1 EventEmitter Pattern

```
const { EventEmitter } = require("events");

class Database extends EventEmitter {
  connect() {
    console.log("connecting...");
    setTimeout(() => {
      this.emit("connected");
    }, 1000);
  }

  query(sql) {
    this.emit("query", { sql, timestamp: Date.now() });
  }
}

const db = new Database();
```

```

db.on("connected", () => {
  console.log("DB connected");
});

db.on("query", (data) => {
  console.log("Query:", data);
});

db.connect();
db.query("SELECT * FROM users");

```

4.2 Error Handling with EventEmitter

```

db.on("error", (err) => {
  console.error("DB error:", err);
});

// Always emit error events, don't throw
db.emit("error", new Error("Connection failed"));

```

5) File Operations

5.1 Synchronous vs Asynchronous

Sync (blocks):

```

const data = fs.readFileSync("file.txt", "utf8");
console.log(data);

```

Async with callback:

```

fs.readFile("file.txt", "utf8", (err, data) => {
  if (err) console.error(err);
  else console.log(data);
});

```

Async with Promise:

```

const fs = require("fs").promises;

async function readFile() {
  try {
    const data = await fs.readFile("file.txt", "utf8");
    console.log(data);
  } catch (err) {
    console.error(err);
  }
}

```

5.2 Directory Operations

```
const fs = require("fs").promises;

async function listFiles(dir) {
  const files = await fs.readdir(dir);
  for (const file of files) {
    const stat = await fs.stat(path.join(dir, file));
    if (stat.isFile()) {
      console.log("file:", file);
    } else if (stat.isDirectory()) {
      console.log("dir:", file);
    }
  }
}

listFiles("./src");
```

6) Express Middleware Pattern

6.1 Middleware Chain

```
const express = require("express");
const app = express();

// Middleware executes in order
app.use((req, res, next) => {
  console.log("1. Request logged");
  next(); // pass to next middleware
});

app.use(express.json());

app.use((req, res, next) => {
  console.log("2. Body parsed");
  next();
});

app.get("/api/users", (req, res) => {
  res.json({ users: [] });
});

// Error handler (must be last)
app.use((err, req, res, next) => {
  console.error(err);
  res.status(500).json({ error: err.message });
});
```

6.2 Custom Middleware

```
function authenticate(req, res, next) {
  const token = req.headers.authorization;
  if (!token) {
    return res.status(401).json({ error: "Unauthorized" });
  }
  req.user = verifyToken(token); // set user on request
  next();
}

app.get("/api/profile", authenticate, (req, res) => {
  res.json({ user: req.user });
});
```

7) Clustering for Multi-Core Utilization

7.1 Cluster Module

```
const cluster = require("cluster");
const os = require("os");
const express = require("express");

if (cluster.isMaster) {
  const numCores = os.cpus().length;
  console.log(`Master ${process.pid} starting workers...`);

  for (let i = 0; i < numCores; i++) {
    cluster.fork(); // spawn worker process
  }

  cluster.on("exit", (worker, code, signal) => {
    console.log(`Worker ${worker.process.pid} died`);
    cluster.fork(); // restart worker
  });
} else {
  // Worker process
  const app = express();
  app.get("/", (req, res) => {
    res.send(`Hello from worker ${process.pid}`);
  });

  app.listen(3000, () => {
    console.log(`Worker ${process.pid} listening on port 3000`);
  });
}
```

Interview explanation:

- use clustering to utilize all CPU cores.

- each worker runs event loop independently.

8) Debugging and Profiling

8.1 Node Debugger

```
node inspect app.js
# breaks at line 1, use commands:
# c - continue
# n - next line
# s - step into
# p variable - print variable
```

8.2 Heap Snapshots for Memory Leaks

```
const v8 = require("v8");
const fs = require("fs");

// create heap snapshot
const snapshot = v8.writeHeapSnapshot();
console.log("Snapshot written to:", snapshot);

// analyze with Chrome DevTools or clinic
```

8.3 Clinic.js for Performance

```
npm install -g clinic
clinic doctor -- node app.js
# visit clinic dashboard to see latency, memory, CPU
```

9) Revision Checklist

- ☐ Know key differences between browser JS and Node.js.
- ☐ Know when to use Worker Threads.
- ☐ Understand `child_process` for spawning.
- ☐ Understand streams and pipe.
- ☐ Understand EventEmitter pattern.
- ☐ Know async file operations.
- ☐ Understand Express middleware chain.
- ☐ Understand clustering for multi-core.
- ☐ Know debugging tools.